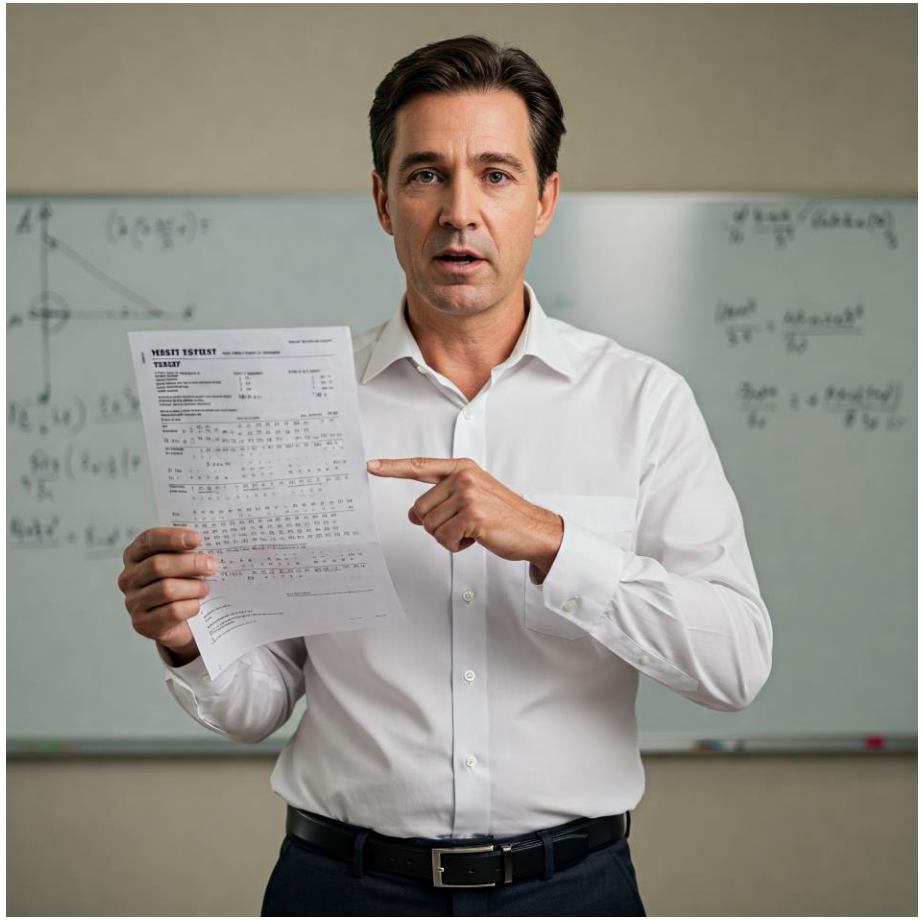
Agenda

- Wyjątki
- Pliki

Wyjątki

Egzamin



```
public class Exam {
  private int length;
  public final Teacher examiner;
  List<Question> questions = new ArrayList<>();
  public Exam(int length, Teacher examinator,
      List<Question> questions) {
    this.length = length;
    this.examinator = examinator;
    this.questions = questions;
  }
public int questionCount() { return questions.size(); }
  public Question getQuestion(int index) {
    return questions.get(index);
  }
}
```

Odpowiedzi – zastosowanie mapy

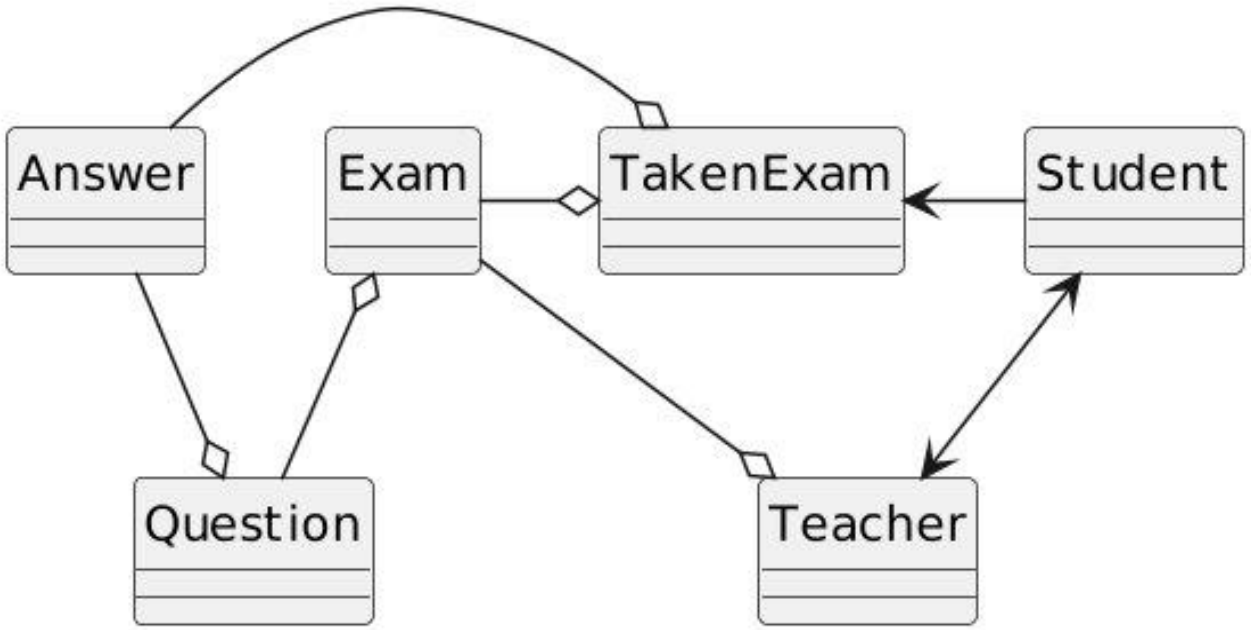
```
public class TakenExam {
  public final Exam exam;
  public final Student student;
  private Map<Question, Answer> answers = new HashMap<>();

  public TakenExam(Student student, Exam exam) {
    this.student = student;
    this.exam = exam;
  }
  public void addAnswer(Answer answer) {
    answers.put(answer.question, answer);
  }
}
```

Pytania i odpowiedzi

```java
public class Question {
  public final String text;
  public Question(String text) { this.text = text; }
}
public class Answer {
  public final Question question;
  private String text;
  public Answer(Question question) {
    this.question = question;
  }
  public void update(String text) {
    this.text = text;
  }
}
```

```java
public class TakenExam {
  public final Exam exam;
  public final Student student;
  private Map<Question, Answer> answers = new HashMap<>();

  public TakenExam(Student student, Exam exam) {
    this.student = student;
    this.exam = exam;
  }
  public void addAnswer(Answer answer) {
    answers.put(answer.question, answer);
  }
}
```

Egzamin – zależności między klasami



Egzamin w toku

```
public class Student {
/* ... */
  void read(Question question) { /* ... */ }
  void update(Answer answer) { /* ... */ }
  boolean validate(Answer answer) { /* ... */ }

  public TakenExam takeExam(Exam exam) {

  }

}
```

Egzamin w toku

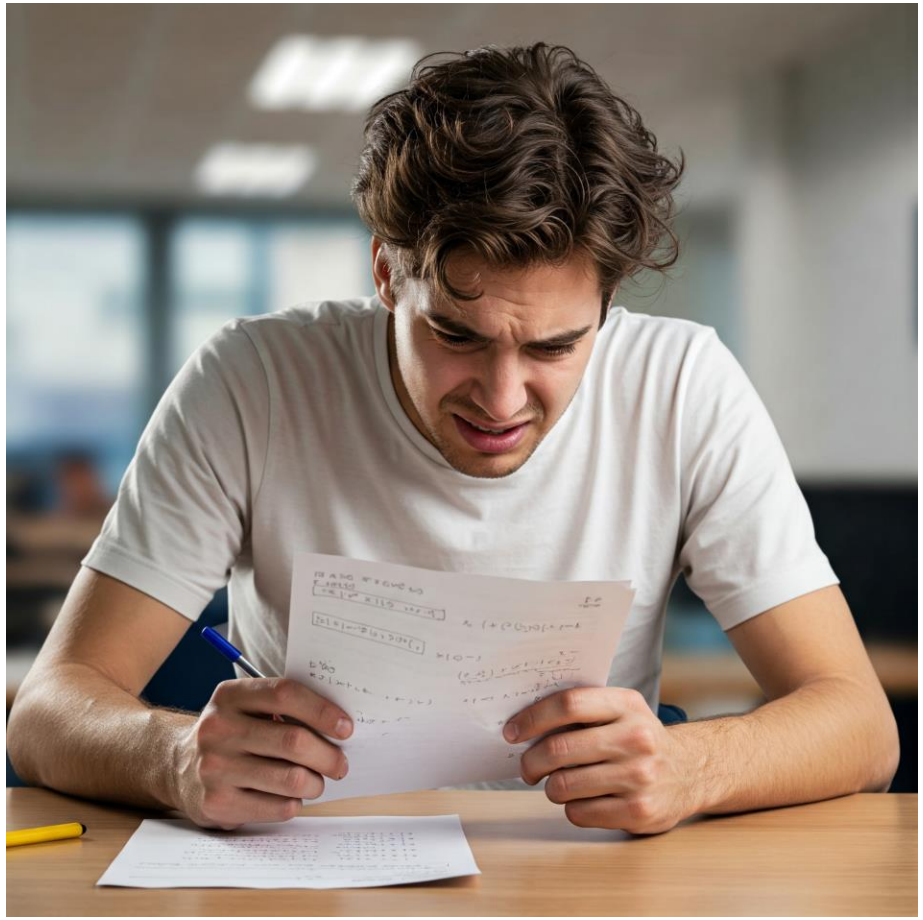
```java
TakenExam takenExam = new TakenExam(this, exam);
for(int i=0; i<exam.questionCount(); ++i) {
  Question question = exam.getQuestion(i);
  Answer answer = new Answer(question);

  do {
    read(question);
    update(answer);
  } while(!validate(answer));

  takenExam.addAnswer(answer);
}
return takenExam;
```

```java
public class Student {
/* ... */
  void read(Question question) { /* ... */ }
  void update(Answer answer) { /* ... */ }
  boolean validate(Answer answer) { /* ... */ }

  public TakenExam takeExam(Exam exam) {

  }

}
```

Mam pytanie!

```
TakenExam takenExam = new TakenExam(this, exam);
for(int i=0; i<exam.questionCount(); ++i) {
  Question question = exam.getQuestion(i);
  Answer answer = new Answer(question);
  do {
    read(question);
    if(needHelp) {
      exam.examiner.ask(question);
      needHelp = false;
    }
    update(answer);
  } while(!validate(answer));
  takenExam.addAnswer(answer);
}
return takenExam;
```

Wyjątek



```
public void suspiciousMethod() {
  doThingsBefore();
  if(somethingFailed()) {
    Exception exception =
              new Exception("Something happened!");
    throw exception;
  }
  doThingsAfter();
}
```

Złapanie wyjątku

```
public void suspiciousMethod() {
  doThingsBefore();
  try {
    if (somethingFailed())
      throw new Exception("Something happened!");
  } catch (Exception e) {
    System.err.println(e.getMessage());
    // Something happened!
  }
  doThingsAfter();
}
```

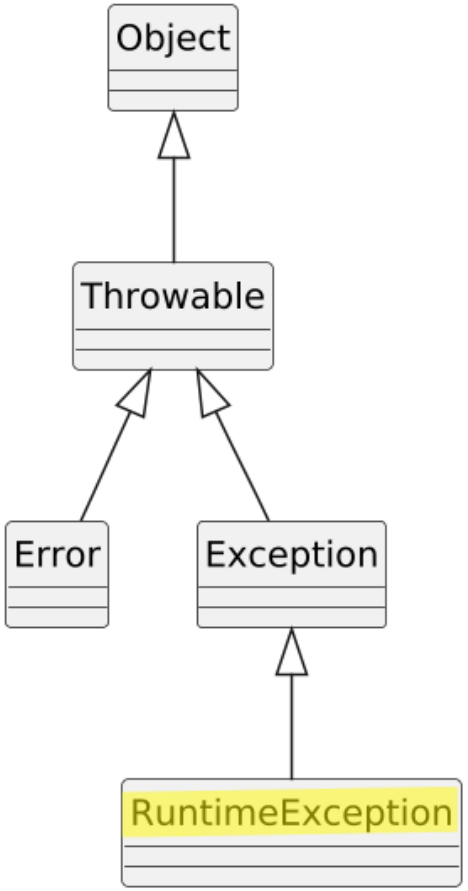
Zrzut stosu

```java
public void suspiciousMethod() {
  doThingsBefore();
  try {
    if (somethingFailed())
      throw new Exception("Something happened!");
  } catch (Exception e) {
    for(var element : e.getStackTrace())
      System.err.println(element);
      // Main.suspiciousMethod(Main.java:5)
      // Main.main(Main.java:21)
  }
  doThingsAfter();
}
```

Przekazanie wyjątku dalej

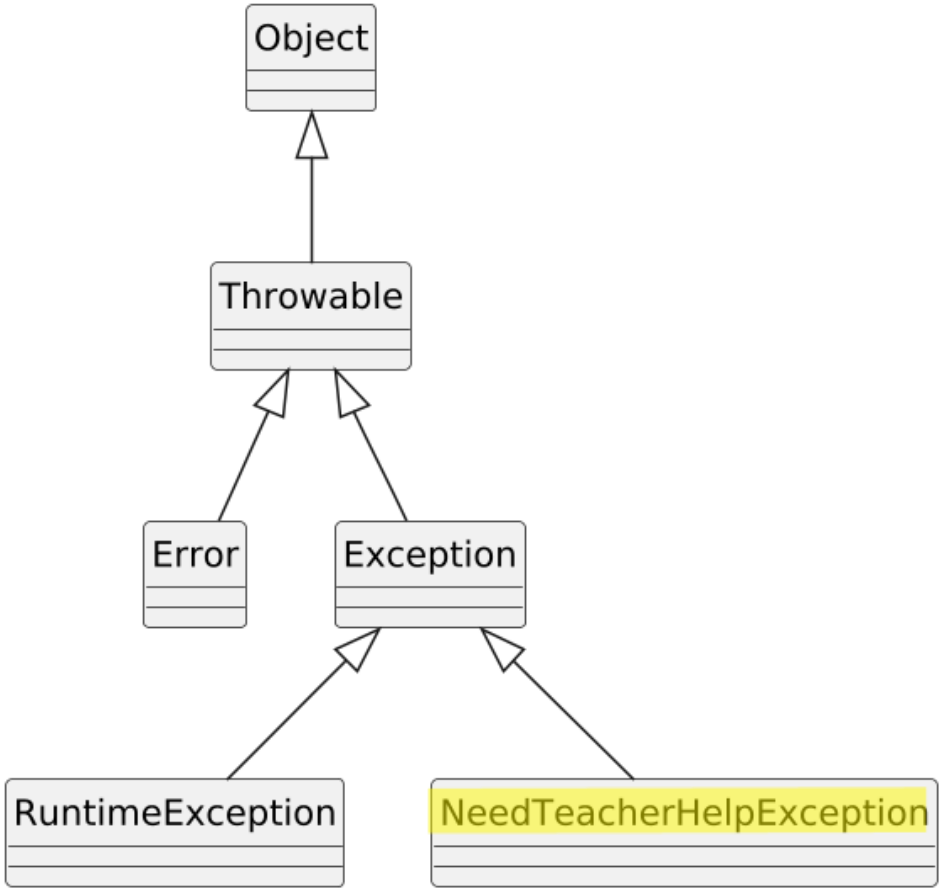
```
public void suspiciousMethod() throws Exception {
  doThingsBefore();
  if (somethingFailed())
    throw new Exception("Something happened!");
  doThingsAfter();
}

public void checkingMethod() {
  try {
    suspiciousMethod();
  } catch (Exception e) {
    System.err.println(e.getMessage());
    // Something happened!
  }
}
```

Wyjątek czasu wykonania




```
public void suspiciousMethod() {
  doThingsBefore();
  if(somethingFailed()) {
   RuntimeException exception =
                 new Exception("Something happened!");
   throw exception;
  }
  doThingsAfter();
}
```

Własne wyjątki




```
public class NeedTeacherHelpException
    extends Exception {
  public NeedTeacherHelpException() {
    super("Need help!");
  }
}

throw new NeedTeacherHelpException();
```

Try, catch

```java
public class Student {
  /* ... */
  private boolean needHelp = true;

  void read(Question question) { /* ... */ }
  void update(Answer answer)
    throws NeedTeacherHelpException {
    if(needHelp)
      throw new NeedTeacherHelpException();
  boolean validate(Answer answer) { /* ... */ }
  public TakenExam takeExam(Exam exam) {
  }
}
```

```java
TakenExam takenExam = new TakenExam(this, exam);
for(int i=0; i<exam.questionCount(); ++i) {
  Question question = exam.getQuestion(i);
  Answer answer = new Answer(question);
  do {
    try {
      read(question);
      update(answer);
    }
    catch (NeedTeacherHelpException exception) {
      exam.examiner.ask(question);
      needHelp = false;
    }
  } while(!validate(answer));
  takenExam.addAnswer(answer);
}
return takenExam;
```

Wiele przechwyceń

```java
public class Student {
  /* ... */
  void read(Question question)
    throws GiveUpException,
    NeedTeacherHelpException {
    /* ... */
  }
  void update(Answer answer)
    throws NeedTeacherHelpException {
    /* ... */
  }
  public TakenExam takeExam(Exam exam) {
  }
}
```

```java
TakenExam takenExam = new TakenExam(this, exam);
for(int i=0; i<exam.questionCount(); ++i) {
  Question question = exam.getQuestion(i);
  Answer answer = new Answer(question);
  do {
    try {
      read(question);
      update(answer);
    }
    catch (NeedTeacherHelpException exception) {
      exam.examier.ask(question);
      needHelp = false;
    }
    catch (GiveUpException exception) { break; }
  } while(!validate(answer));
  takenExam.addAnswer(answer);
}
return takenExam;
```

Zagnieżdżanie bloków try, catch

```
public TakenExam takeExam(Exam exam) {
  TakenExam takenExam = new TakenExam(this, exam);
    for(int i=0; i<exam.questionCount(); ++i) {
      Question question = exam.getQuestion(i);
      Answer answer = new Answer(question);
    try {
    }
    catch(GiveUpException ignored) { }
    takenExam.addAnswer(answer);
  }
  return takenExam;
}
```

```
do {
  try {
    read(question);
    update(answer);
  } catch (NeedTeacherHelpException exception) {
    exam.examinator.ask(question);
  }
} while (!validate(answer));
```

Blok finally

```java
public TakenExam takeExam(Exam exam) {
  TakenExam takenExam = new TakenExam(this, exam);
    for(int i=0; i<exam.questionCount(); ++i) {
      Question question = exam.getQuestion(i);
      Answer answer = new Answer(question);
    try {
    }
    catch(GiveUpException ignored) { }
    finally {
      takenExam.addAnswer(answer);
    }
  }
  return takenExam;
}
```

```java
do {
  try {
    read(question);
    update(answer);
  } catch (NeedTeacherHelpException exception) {
    exam.examinator.ask(question);
  }
} while (!validate(answer));
```

Długość egzaminu

```java
public class Exam {
  private int length;
  private LocalTime startTime;

  /* ... */
  public void start() {
    startTime = LocalTime.now();
  }
  public long timeLeft() {
    return ChronoUnit.MINUTES.between(startTime, LocalTime.now());
  }
}
```

```java
public class MakeHasteException extends Exception {
  public MakeHasteException() {
    super("Go go go!");
  }
}
```

Kończy się czas!

```
public class Student {
  private static final long minTimeForQuestion =
  10;
  private void checkTime(Exam exam, int
  questionsLeft) throws MakeHasteException {
    long timeForEachQuestion = exam.timeLeft() /
    questionsLeft;
    if(timeForEachQuestion < minTimeForQuestion)
      throw new MakeHasteException();
  }
  public TakenExam takeExam(Exam exam) {
  }
}
```

```
TakenExam takenExam = new TakenExam(this, exam);
for(int i=0; i<exam.questionCount(); ++i) {
  Question question = exam.getQuestion(i);
  Answer answer = new Answer(question);
  try {
    do {
      try {
        read(question); update(answer);
        checkTime(exam, exam.questionCount() - i);
      } catch (NeedTeacherHelpException exception) {
        exam.examinator.ask(question);
      }
    } while (!validate(answer));
  }
  catch(GiveUpException ignored) { }
  catch(MakeHasteException ignored) { }
  finally { takenExam.addAnswer(answer); }
}
return takenExam;
```

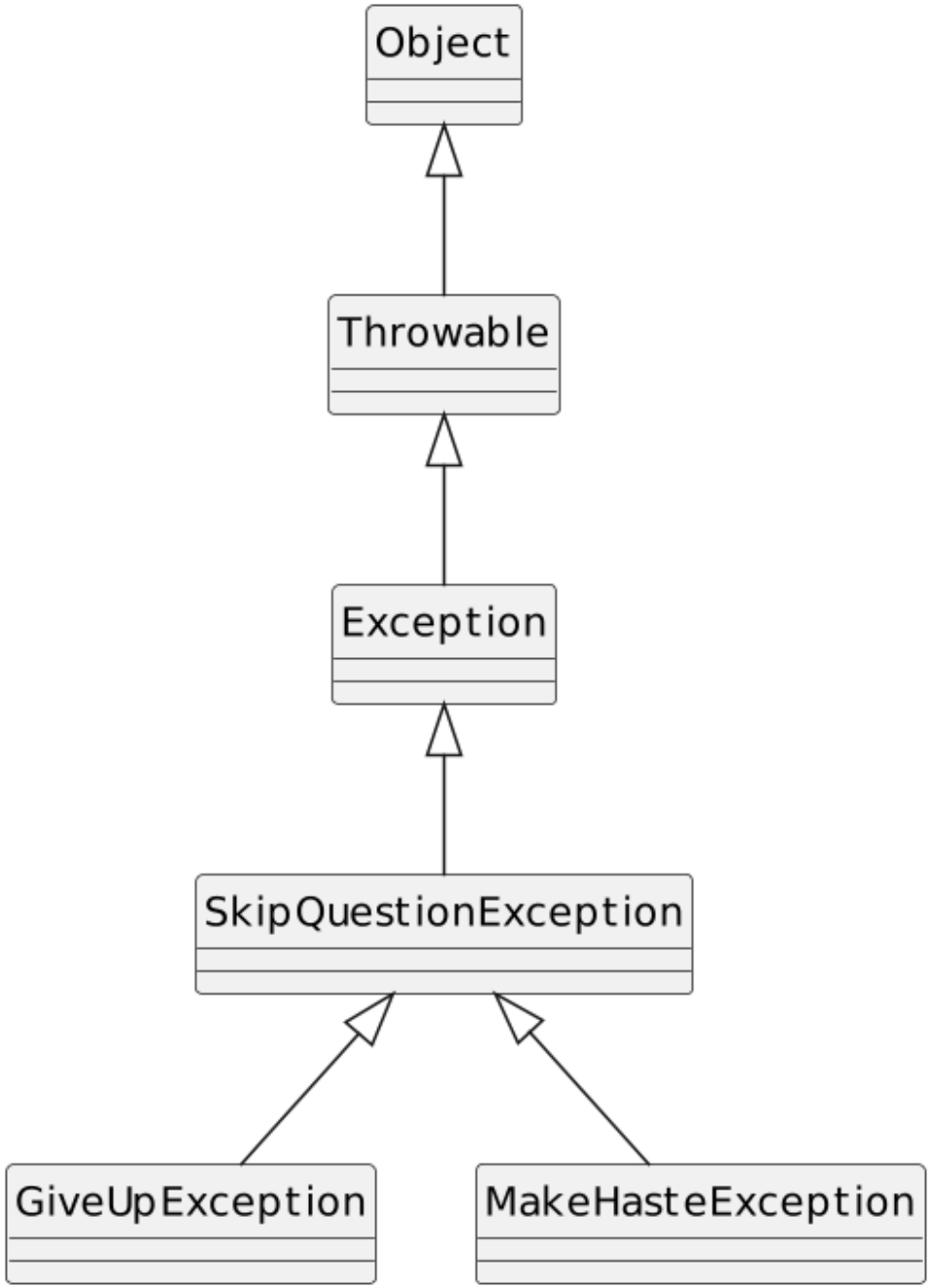
Łączenie duplikatów

```java
public class Student {
  private static final long minTimeForQuestion = 10;
  private void checkTime(Exam exam, int questionsLeft) throws MakeHasteException {
    long timeForEachQuestion = exam.timeLeft() / questionsLeft;
    if(timeForEachQuestion < minTimeForQuestion)
      throw new MakeHasteException();
  }
  public TakenExam takeExam(Exam exam) {
  }
}
```

```java
TakenExam takenExam = new TakenExam(this, exam);
for(int i=0; i<exam.questionCount(); ++i) {
  Question question = exam.getQuestion(i);
  Answer answer = new Answer(question);
  try {
    do {
      try {
        read(question); update(answer);
        checkTime(exam, exam.questionCount() - i);
      } catch (NeedTeacherHelpException exception) {
        exam.examinator.ask(question);
      }
    } while (!validate(answer));
  }
  catch(GiveUpException
    | MakeHasteException ignored) { }

  finally { takenExam.addAnswer(answer); }
}
return takenExam;
```

Dziedziczenie wyjątków

```
public abstract class SkipQuestionException extends Exception {
  SkipQuestionException(String message) {
    super(message);
  }
}
public class GiveUpException extends SkipQuestionException {
  public GiveUpException() {
    super("I'm done with it");
  }
}
public class MakeHasteException extends SkipQuestionException {
  public MakeHasteException() {
    super("Go go go!");
  }
}
```



Polimorfizm wyjątków

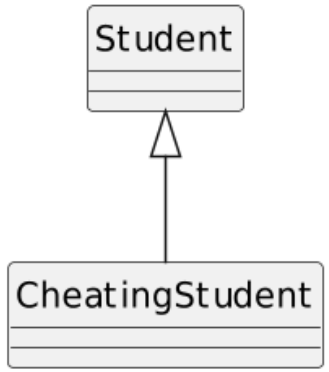
```java
class Student {
  private void read(Question question) throws
  NeedTeacherHelpException, GiveUpException {
    /* ... */
}
  private void update(Answer answer) throws
  NeedTeacherHelpException {
    /* ... */
}
  private void checkTime(Exam exam,
  int questionsLeft) throws MakeHasteException {
    /* ... */
}
  public TakenExam takeExam(Exam exam) {

  }
}
```

```java
TakenExam takenExam = new TakenExam(this, exam);
for(int i=0; i<exam.questionCount(); ++i) {
  Question question = exam.getQuestion(i);
  Answer answer = new Answer(question);
  try {
    do {
      try {
        read(question); update(answer);
        checkTime(exam, exam.questionCount() - i);
      } catch (NeedTeacherHelpException exception) {
        exam.examinator.ask(question);
      }
    } while (!validate(answer));
  }
  catch(SkipQuestionException ignored) { }

  finally { takenExam.addAnswer(answer); }
}
return takenExam;
```

Oszuści

```
public class CheatingStudent extends Student {
  private boolean cheating;
  Random generator = new Random();
  float shamelessness;

  public boolean isCheating() {
    return cheating;
  }
  public CheatingStudent(float shamelessness) {
    this.shamelessness = shamelessness;
  }

}
```

Oszuści




```java
public class CheatingStudent extends Student {
  private boolean cheating;
  Random generator = new Random();
  float shamelessness;

  public boolean isCheating() {
    return cheating;
  }
  public CheatingStudent(float shamelessness) {
    this.shamelessness = shamelessness;
  }

}
```

```java
@Override
void update(Answer answer) throws
   NeedTeacherHelpException {
  super.update(answer);
  cheating = generator.nextFloat() > shamelessness;
}
```

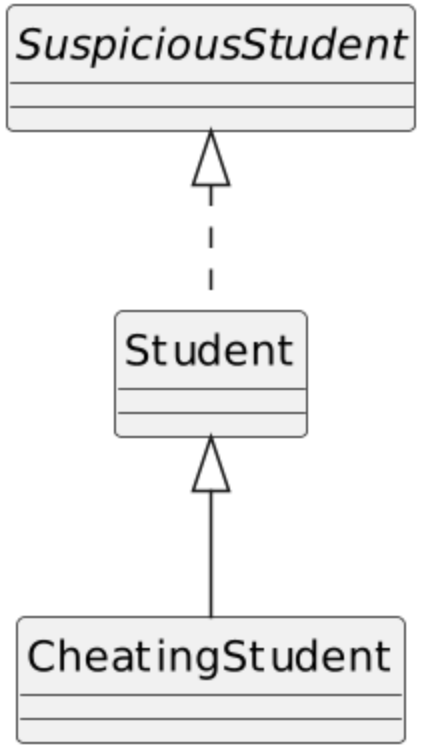
Poza podejrzeniami?

```java
public class CheatingStudent extends Student {
  private boolean cheating;
  Random generator = new Random();
  float shamelessness;

  public boolean isCheating() {
    return cheating;
  }
  public CheatingStudent(float shamelessness) {
    this.shamelessness = shamelessness;
  }
  /* ... */
}
```

```java
public class Teacher {
  void ask(Question question) {
    /* ... */
  }

  boolean control(Student student) {
    return student.isCheating();
  }
}
```

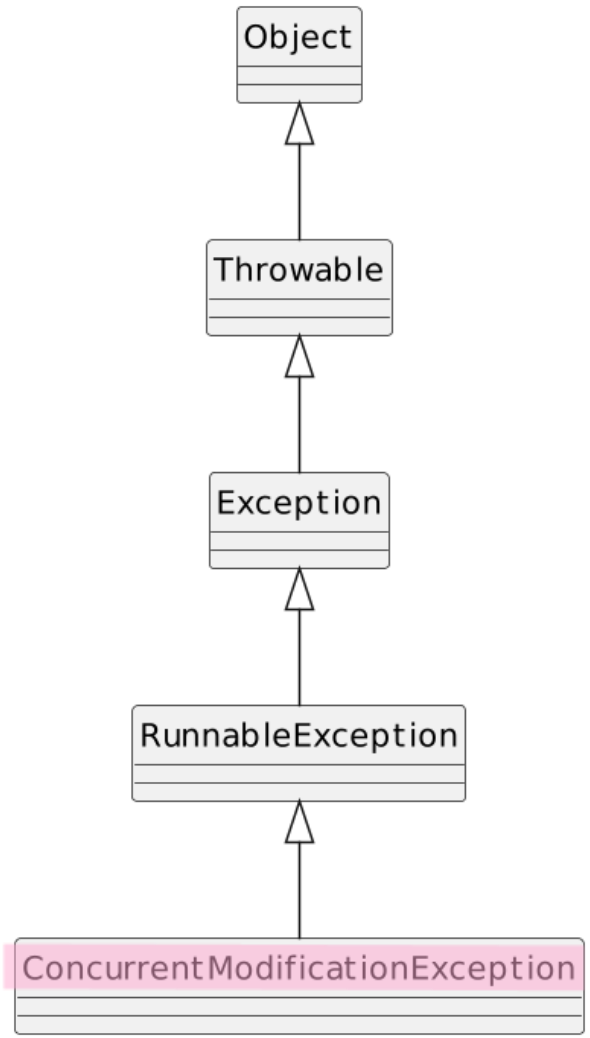
Wszyscy są podejrzani

```
interface SuspiciousStudent {
  default boolean isCheating() {
    return false;
  }
}
public class Student implements SuspiciousStudent {
  /* ... */
}
public class CheatingStudent extends Student {
  /* ... */
  public boolean isCheating() {
    return cheating;
  }
}
```



Nie tak łatwo się ich pozbyć

```
public class Teacher {
  void ask(Question question) {
    /* ... */
  }

  void control(List<Student> students) {
    for (Student student : students)
      if (student.isCheating())
        students.remove(student);
  }
}
```

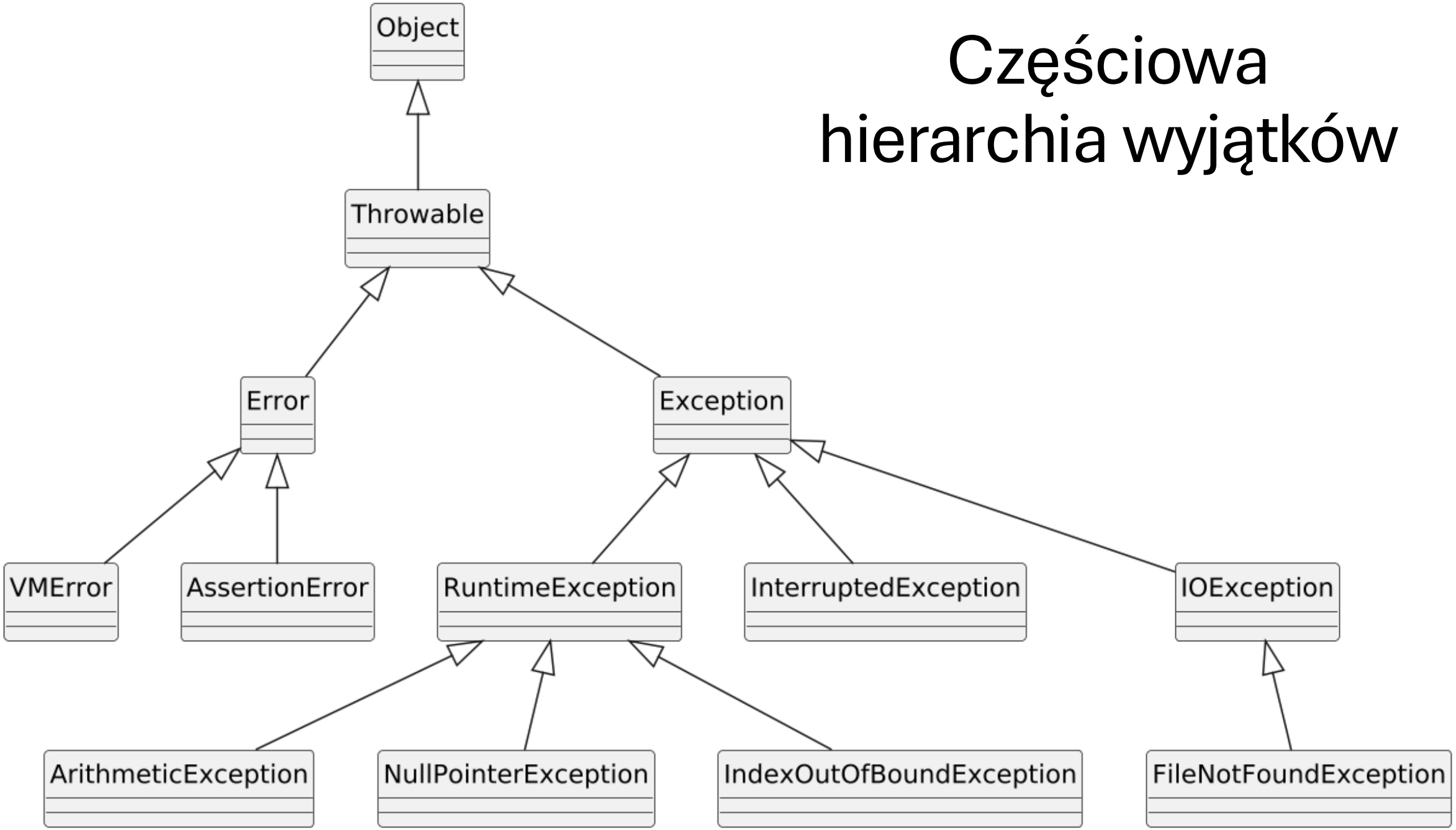


Wyjątek z polami

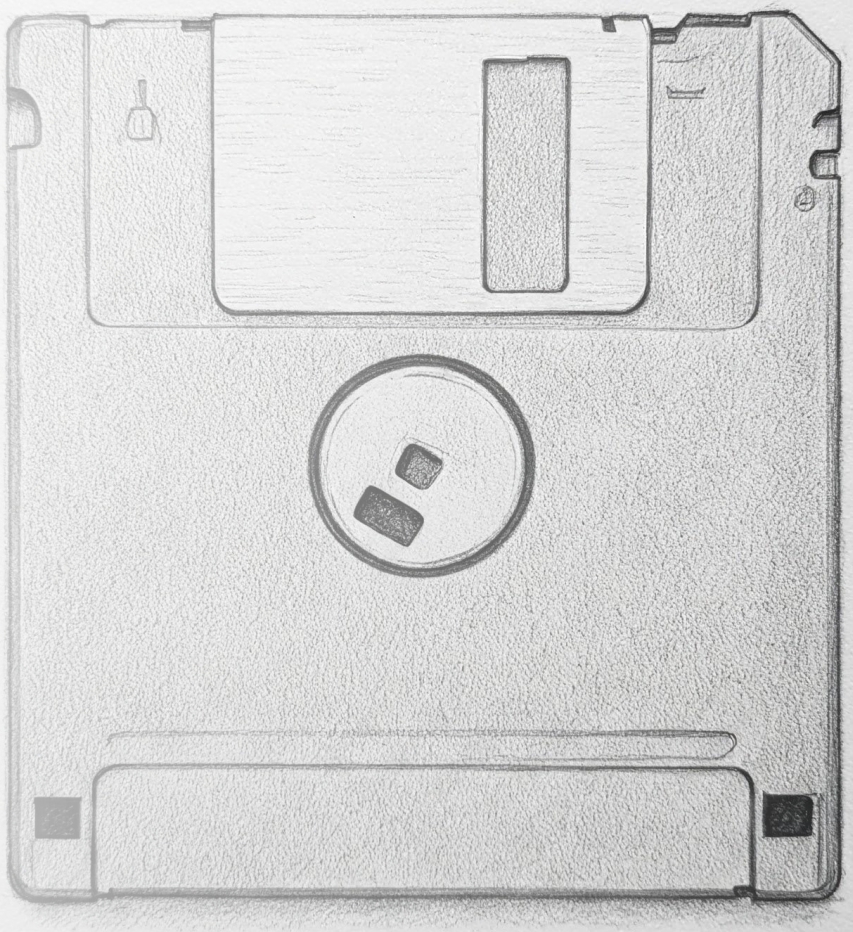
```java
class CatchCheaterException extends Exception {
  public final Student student;
  public CatchCheaterException(Student student) {
    super("Catch cheating student");
    this.student = student;
  }
}
```

```java
class Teacher {
  /* ... */
  void control(List<Student> students) {
    try {
      for (Student student: students)
        if (student.isCheating())
          throw new CatchCheaterException(student);
    }
    catch (CatchCheaterException exception) {
      students.remove(exception.student);
      control(students);
    }
}
```

Częściowa hierarchia wyjątków



Pliki



Pliki tekstowe

Zapis:

- FileWriter
- PrintWriter
- BufferedWriter

Odczyt:

- FileReader
- BufferedReader
- Scanner

FileWriter

```java
File file = new File("out.txt");
try {
  FileWriter writer = new FileWriter(file);
  // append: FileWriter writer = new FileWriter(file, true);
  writer.write("Simple one-liner");
  writer.close();
} catch (IOException e) {
  System.err.println("Cannot access: " + file.getName());
}
```

PrintWriter

```
File file = new File("out.txt");
try {
  PrintWriter writer = new PrintWriter(file);
  writer.print("First line\n");
  writer.println("Second line");
  writer.printf("%s line", "Third\n");
  writer.close();
} catch (FileNotFoundException e) {
  System.err.println("Cannot open: " + file.getName());
}
```

BufferedWriter

```
File file = new File("out.txt");
try {
  BufferedWriter writer = new BufferedWriter(new FileWriter(file));
  for(int i=0; i < 10; ++i) {
    for (int j = 0; j < 1000; ++j)
      writer.write("X");
    writer.flush();
  }
  writer.close();
} catch (IOException e) {
  System.err.println("Cannot access: " + file.getName());
}
```

Korzyść z buforowania

```
File file = new File("out.txt");
try {
BufferedWriter writer = new BufferedWriter(new
  FileWriter(file));
  long start = System.currentTimeMillis();
  for(int i=0; i < 100000000; ++i)
    writer.write("X");
long finish = System.currentTimeMillis();
System.out.println(finish-start);
writer.close();
} catch (IOException e) {
throw new RuntimeException(e);
}
// 992
```

```
File file = new File("out.txt");
try {
  FileWriter writer = new FileWriter(file);

  long start = System.currentTimeMillis();
  for(int i=0; i < 100000000; ++i)
    writer.write("X");
long finish = System.currentTimeMillis();
System.out.println(finish-start);
writer.close();
} catch (IOException e) {
throw new RuntimeException(e);
}
// 2246
```

FileReader

```
File file = new File("in.txt");
String result = "";
try {
  FileReader reader = new FileReader(file);
  int next;
  while ((next = reader.read()) != -1)
    result += (char) next;
  reader.close();
} catch (IOException e) {
  System.err.println("Cannot access: " + file.getName());
}
System.out.println(result);
```

StringBuilder

```
File file = new File("in.txt");
StringBuilder result = new StringBuilder();
try {
  FileReader reader = new FileReader(file);
  int next;
  while ((next = reader.read()) != -1)
    result.append((char) next);
  reader.close();
} catch (IOException e) {
  System.err.println("Cannot access: " + file.getName());
}
System.out.println(result);
```

BufferedReader

```
File file = new File("in.txt");
StringBuilder result = new StringBuilder();
try {
  BufferedReader reader = new BufferedReader(new FileReader(file));
  String line;
  while ((line = reader.readLine()) != null)
    result.append(line).append("\n");
  reader.close();
} catch (IOException e) {
  System.err.println("Cannot access: " + file.getName());
}
System.out.println(result);
```

Scanner

```
File file = new File("in.txt");
StringBuilder result = new StringBuilder();
try {
  Scanner scanner = new Scanner(file);
  scanner.useDelimiter(" ");
  while(scanner.hasNext())
    result.append(scanner.next()).append(" ");
  scanner.close();
} catch (FileNotFoundException e) {
  System.err.println("Cannot access: " + file.getName());
}
System.out.println(result);
```

Pliki binarne

Zapis:

- FileOutputStream
- DataOutputStream

Odczyt:

- DataInputStream

FileOutputStream

```
File file = new File("out.bin");
byte[] data = {1, 2, 3, 4, 5};
try {
  FileOutputStream fos = new FileOutputStream(file);
  fos.write(data);
  fos.close();
} catch (IOException e) {
  System.err.println("Cannot access: " + file.getName());
}
```

FileOutputStream

```
File file = new File("out.bin");
int[] data = {1, 2, 3, 4, 5};
try {
  FileOutputStream fos = new FileOutputStream(file);
  for (int i = 0; i < data.length; i++) {
    fos.write(data[i] >> 24);  fos.write(data[i] >> 16);
    fos.write(data[i] >> 8);   fos.write(data[i]);
  }
  fos.close();
} catch (IOException e) {
  System.err.println("Cannot access: " + file.getName());
}
```

DataOutputStream

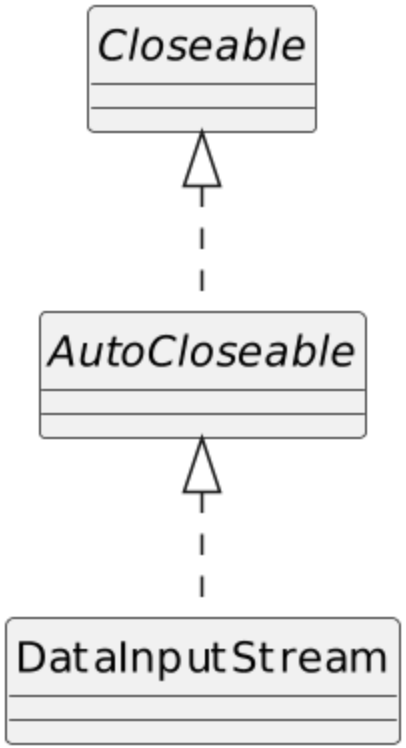
```
File file = new File("out.bin");
byte[] data = {1, 2, 3, 4, 5};
try {
  FileOutputStream fos = new FileOutputStream(file);
  DataOutputStream dos = new DataOutputStream(fos);
  for (int i = 0; i < data.length; i++)
    dos.writeInt(data[i]);

  fos.close();
} catch (IOException e) {
  System.err.println("Cannot access: " + file.getName());
}
```

DataInputStream

```
File file = new File("in.bin");
try {
  FileInputStream fis = new FileInputStream(file);
  DataInputStream dis = new DataInputStream(fis);
  int size = (int) (file.length() / Integer.BYTES);
  int[] data = new int[size];
  for(int i=0; i<size; ++i)
    data[i] = dis.readInt();
  for(int value: data) System.out.println(value);
  dis.close();
} catch (IOException e) {
  System.err.println("Cannot access: " + file.getName());
}
```

Try-with-resources

```
File file = new File("in.bin");
try (
  FileInputStream fis = new FileInputStream(file);
  DataInputStream dis = new DataInputStream(fis);
) {
  int size = (int) (file.length() / Integer.BYTES);
  int[] data = new int[size];
  for(int i=0; i<size; ++i)
    data[i] = dis.readInt();
  for(int value: data) System.out.println(value);
} catch (IOException e) {
  System.err.println("Cannot access: " + file.getName());
}
```



Serializacja obiektów

Zapis:

- ObjectOutputStream

Odczyt:

- ObjectInputStream

Interfejs Serializable

```
class Exam implements Serializable {
  private int length;
  private LocalTime startTime;
  public final Teacher examiner;
  private List<Question> questions = new ArrayList<>();
 }
```

Interfejs Serializable

```
class Exam implements Serializable {
  private int length;
  private LocalTime startTime;
  public final Teacher examiner;
  private List<Question> questions = new ArrayList<>();
 }
```

ObjectOutputStream

```
Exam exam = new Exam(60, teacher, questions);
File file = new File("out.bin");
try (
  FileOutputStream fos = new FileOutputStream(file);
  ObjectOutputStream oos = new ObjectOutputStream(fos);
) {
  oos.writeObject(exam);
} catch (IOException e) {
  System.err.println("Cannot access: " + file.getName());
}
```

ObjectInputStream

```
Exam exam = null;
File file = new File("out.bin");
try (
  FileInputStream fis = new FileInputStream(file);
  ObjectInputStream ois = new ObjectInputStream(fis);
) {
  exam = (Exam) ois.readObject();
} catch (IOException e) {
  System.err.println("Cannot access: " + file.getName());
} catch (ClassNotFoundException e) {
  System.err.println("No objects found in: " + file.getName());
}
```